

Guide détaillé : Comment réussir les partiels Programmation Générique et Programmation Objet (Annales 2023 - 2025)

Ce document explique la méthode à suivre pendant les partiels de PGO. L'objectif est de comprendre : - comment analyser le sujet, - quelles structures STL choisir, - comment écrire les classes, - comment rédiger rapidement un code correct, - comment gagner des points même si le code n'est pas parfait.

1. Méthode générale pour TOUS les sujets

Étape 1 : Lire le sujet complètement

Toujours lire les deux parties avant de commencer.
Les exercices sont souvent indépendants.

Étape 2 : Repérer les structures STL demandées

Le professeur demande presque toujours :

- vector
- map
- set
- priority_queue
- pair
- sort
- count_if
- remove_if

Il faut immédiatement identifier :

- quelles données stocker,
- si les clés sont uniques,
- si un tri est nécessaire,
- si l'ordre compte.

Étape 3 : Commencer par les classes

Toujours écrire :
- attributs privés,
- constructeurs,
- getters/setters,
- opérateurs STL si nécessaire.

Étape 4 : Ajouter les fonctions demandées

Le sujet demande souvent :

- ajouter,
- supprimer,
- rechercher,
- afficher,
- compter.

Étape 5 : Faire un main() de test

Même simple.
Le professeur veut voir :
- création d'objets,
- appels de fonctions,
- affichage.

Conseil important :

Même si le code n'est pas parfait :
- écrire les structures STL,
- écrire les signatures,
- écrire les classes,
fait déjà gagner beaucoup de points.

2. Comment faire les sujets sur vector

```
Quand utiliser vector ?
-----
Utiliser vector quand :
- on stocke une liste,
- l'ordre est important,
- on veut parcourir facilement.

Exemple typique :
-----
std::vector<Livres> livres;

Méthodes importantes :
-----
push_back()
begin()
end()
size()

Exemple :
-----
livres.push_back(l);

Parcours :
-----
for(auto l : livres) {
    std::cout << l.getTitre();
}

Suppression :
-----
Utiliser remove_if + erase.

Exemple :
-----
livres.erase(
    remove_if(...),
    livres.end()
);
```

3. Comment faire les sujets sur map

```
Quand utiliser map ?
-----
Quand il existe :
- une clé unique,
- une valeur associée.

Exemple :
-----
matricule -> informations

Déclaration :
-----
std::map<Cle, Valeur> table;

Pourquoi map est très utilisé en examen ?
-----
Car :
- tri automatique,
- accès rapide,
- parfait pour dictionnaires/fréquences.

Exemple fréquences :
-----
std::map<std::string,int> freq;

freq[mot]++;

Parcours :
-----
for(auto it : freq) {
    std::cout << it.first
               << it.second;
}
```

4. Comment faire les sujets sur les fréquences

Sujet classique :

Le professeur donne :
- un texte,
- un fichier,
- des mots.

Objectif :

Compter les apparitions.

Méthode :

1. Lire le fichier.
2. Stocker les mots dans vector.
3. Utiliser map<string,int>.
4. Faire freq[mot]++.

Lecture fichier :

std::ifstream f("texte.txt");

std::string mot;

while(f >> mot) {
 mots.push_back(mot);
}

Calcul :

for(auto m : mots) {
 freq[m]++;
}

Très important :

Les exercices de fréquence utilisent presque toujours :
- map,
- vector,
- set,
- sort.

5. Comment faire les classes rapidement

Structure standard :

class Livre {
private:
 attributs;
public:
 constructeur;
 getters;
 setters;
};

Méthode rapide :

1. Lire les données du sujet.
2. Chaque donnée devient un attribut.
3. Faire le constructeur.
4. Ajouter getters/setters.

Exemple :

titre -> string
année -> int
id -> int
Donc :

```

-----
std::string titre;
int annee;
int id;

```

6. Comment choisir les algorithmes STL

```

sort :
-----
Pour trier.

count_if :
-----
Pour compter avec une condition.

remove_if :
-----
Pour supprimer.

find_if :
-----
Pour rechercher.

Exemple count_if :
-----
count_if(
    livres.begin(),
    livres.end(),
    [](Livres l) {
        return l.getAnnee() > 2000;
    }
);

Important :
-----
Le professeur adore les lambda expressions.

```

7. Comment faire les sujets sur priority_queue

```

priority_queue sert à :
-----
Traiter les éléments par priorité.

Exemple :
-----
urgences écologiques.

Déclaration :
-----
std::priority_queue<Urgence> file;

Important :
-----
Il faut redéfinir operator<.

Exemple :
-----
bool operator<(const Urgence& u) const {
    return niveau < u.niveau;
}

Résultat :
-----
La plus grosse priorité sort d'abord.

Méthodes :
-----
push()
top()
pop()
empty()

```

8. Stratégie pendant l'examen

Ordre conseillé :

1. Structures STL.
2. Classes.
3. Constructeurs.
4. Fonctions simples.
5. Fonctions STL.
6. main().

Si bloqué :

Ne jamais laisser vide.

Écrire :

- les attributs,
- les signatures,
- les boucles,
- les commentaires.

Même incomplet = points.

Très important :

Les examens PGO évaluent surtout :

- la logique objet,
- le choix STL,
- la structure du code.

Le code parfait n'est pas obligatoire pour avoir une bonne note.